

CIFAR-10 Classification Challenge

Marc Germanos, Parsa Jafarian, Nicolas Saade
Electrical and Computer Engineering, McGill University, Montreal, Canada
{parsa.jafarian, nicolas.saade, marc.germanos}@mail.mcgill.ca

Abstract—This study compares Vision Transformers (ViT), Convolutional Neural Networks (CNN), and ensemble models for image classification on a dataset of 10,000 RGB images of size 32×32 , spanning 10 classes of animals and vehicles. A custom ViT configuration achieved 69.88% training accuracy and 59.50% test accuracy after 50 epochs, with its main limitation arising from overfitting on the small dataset and weaker performance on animal classes compared to vehicles. In contrast, a deep residual CNN with 7 blocks achieved around 82% test accuracy, demonstrating superior generalization due to its inductive biases. Finally, an ensemble of three fine-tuned architectures (ConvNeXt, ResNet-v2, Swin ViT) achieved 97.83% test accuracy on Kaggle, highlighting the effectiveness of ensembling and fine-tuning techniques in leveraging diverse inductive biases for state-of-the-art performance.

Index Terms—Image Classification, Vision Transformer, Convolutional Neural Networks, Ensemble Learning

I. INTRODUCTION

The primary objective of this work is to design and train two image-classification models from scratch: a Convolutional Neural Network (CNN) and a Vision Transformer (ViT). A secondary objective is to train and evaluate a third, unrestricted model (potentially pre-trained) for participation in the course Kaggle competition, with the aim of achieving the highest possible leaderboard accuracy.

A. Multi-class classification

Formally, the goal of the classifiers is to assign each input image to exactly one of K possible classes. To achieve this, the model produces a vector of real-valued scores (logits), which are converted into a probability distribution using the softmax function [1].

Formally, given logits $\mathbf{z} = [z_1, \dots, z_K]$, the softmax function is defined as:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, \dots, K. \quad (1)$$

To train the model, we use the Cross-Entropy loss, which measures the discrepancy between the predicted probabilities and the true class label [1]. Given a one-hot encoded target vector $\mathbf{y}$, the loss is:

$$\mathcal{L}(\mathbf{z}, \mathbf{y}) = - \sum_{i=1}^K y_i \log(\text{softmax}(\mathbf{z})_i). \quad (2)$$

For regularization, label smoothing can be applied to the target vector $\mathbf{y}$ before it is passed into the Cross-Entropy loss function. Instead of using a strict one-hot encoding, label

smoothing replaces the empirical target distribution with a softened version:

$$\tilde{\mathbf{y}} = (1 - \epsilon)\mathbf{y} + \frac{\epsilon}{K}\mathbf{1}, \quad (3)$$

where ϵ is the smoothing parameter, and $\mathbf{1}$ denotes a vector of ones [2]. Label smoothing regularizes training by softening one-hot targets, redistributing a small fraction of probability mass uniformly across all classes, which reduces overconfidence, improves generalization, and calibrates predicted probabilities.

Beyond loss, we also evaluate model performance using classification accuracy, which measures the proportion of correctly predicted samples. If $\hat{y}$ is the predicted class and y is the true class, accuracy is computed as:

$$\text{Accuracy} = \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \mathbb{I}[\hat{y}_i = y_i], \quad (4)$$

where $\mathcal{B}$ is a batch of the dataset and $\mathbb{I}[\cdot]$ is the indicator function that equals 1 when the prediction is correct and 0 otherwise.

B. Dataset and Results

Experiments were conducted on a dataset of 10,000 RGB images of size 32×32 , spanning 10 classes of animals and vehicles.

The Vision Transformer (ViT) partitions an image into fixed-size patches and applies self-attention to capture relationships among them. A custom configuration achieved the strongest results among tested variants, reaching 69.88% training accuracy and 59.50% test accuracy after 50 epochs. While these results demonstrate the promise of transformers for image classification, they also reveal challenges such as overfitting on small datasets and weaker performance on animal classes compared to vehicles, reflecting the data-hungry nature of ViTs and their limited inductive biases relative to CNNs.

For the Convolutional Neural Networks (CNN) it employed a deep residual architecture to extract features, utilizing residual connections to facilitate the training of a 7 block deep model. The best configuration demonstrated a testing accuracy of around 82%. These results surpass the Vision Transformer on this task. This highlights the advantage of the CNN's inductive biases

The model ensemble is composed of three different model architectures fine-tuned on our dataset (ConvNeXt, ResNet-v2, Swin ViT), giving us a diverse set of inductive biases

to aggregate our results across our 3 models, our ensemble achieved a test set accuracy of 97.83% on Kaggle. Highlighting the advantage of ensembling and fine-tuning techniques used [3], [4], [5].

II. DATASET

The dataset comprises 10,000 RGB images of size 32×32 , each annotated with one of 10 distinct classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Representative examples from each class are illustrated in Fig. 1. Accordingly, any batch of images $\mathcal{B}$ can be expressed with dimensions $(|\mathcal{B}|, C, H, W)$, where $C = 3$ corresponds to the color channels, and $H = W = 32$ specify the image height and width, respectively.

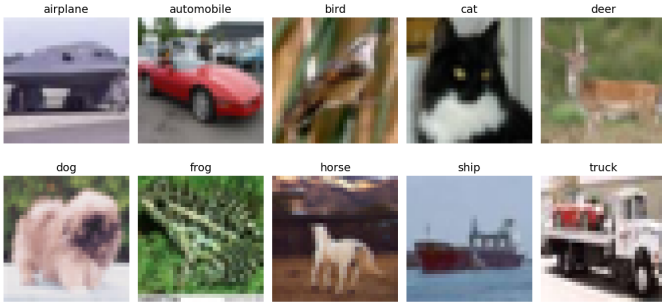


Fig. 1. Representative samples from the dataset, illustrating the 10 target classes for classification.

For the convolutional neural network (CNN), 32×32 images were resized to 160×160 , randomly cropped to 128×128 , and augmented with horizontal flips, rotations up to $\pm 15^\circ$, affine translations (10%), and color jitter. Images were converted to tensors, normalized (mean=0.5, std=0.5), and trained with batch size 64. Generalization was enhanced using a deterministic "Alternating Flip" strategy and Mixup/CutMix (p=0.7).

For the vision transformer (ViT), preprocessing was simpler: images were resized and cropped to 128×128 , augmented only with horizontal flips, then converted to tensors, normalized identically, and trained with batch size 128.

Validation for both models used deterministic resizing to 128×128 and normalization without augmentation.

For the ensemble, preprocessing pipelines were subsets of the CNN's, adapted per architecture. Mixup and CutMix were excluded, while AutoAugment (CIFAR-10 policy) and Random Erasing (p=0.25) were applied for Swin-ViT.

III. METHODOLOGY

A. Convolutional Neural Network

a) Architecture: Our CNN classifier processes input images through a sequence of L residual blocks, as illustrated in Fig. 2. Each residual block consists of:

- **Main path:** two 3×3 convolutional layers, each followed by batch normalization and a GELU activation function. A dropout layer is applied after the first convolution to regularize the network.

- **Skip path:** the input is added to the main path output. If the input and output channel dimensions differ, a 1×1 convolution projects the input to the correct dimension before addition.

In our specific implementation, we utilized a depth of $L = 7$ blocks. The network progressively increases its feature representation capacity by expanding the channel width (number of neurons) at each stage. The channel dimensions for the sequential blocks are set to [32, 64, 128, 256, 512, 512, 512]. Consequently, the feature maps are spatially down-sampled via max pooling after every block, while the depth increases, compressing spatial information into rich semantic features.

After addition, a final GELU activation is applied, and the output is down-sampled using 2×2 max pooling. Once all L residual blocks have been applied, the resulting feature map is globally pooled using an adaptive average pooling layer [6], flattened, and passed through a fully connected linear layer to produce the final class logits. These logits are then passed into Cross-Entropy loss (2).

b) Convolution: The central element of this architecture is the convolutional layer. Images have multiple channels—in our case three—which introduce an additional dimension in the convolution operation. At times, it is necessary to project an input channel C_{in} into an output channel C_{out} . To achieve this, the kernel $\mathbf{W}$ is represented as a 4D tensor $\mathbf{W}_{u,v,c,d}$, where u, v index the spatial dimensions of the kernel, c indexes the input channel, and d indexes the output channel. The convolution is then defined as

$$[\mathbf{W} * \mathbf{X}](i, j, d) = b + \sum_{u=0}^k \sum_{v=0}^k \sum_{c=0}^{C_{in}} x_{si+u, sj+v, c} w_{u,v,c,d}, \quad (5)$$

where s denotes the stride of the kernel, i.e., the step size with which the kernel $\mathbf{W}$ moves across the spatial dimensions of the input $\mathbf{X}$, and b is the bias associated with the output channel d [7]. This operation extracts spatial features by emphasizing relevant patterns such as edges or textures, as illustrated in Fig. 3.

c) Pooling: Convolutional layers are translation-equivariant, but pooling introduces invariance by summarizing local regions. Max pooling takes the strongest activation, while average pooling computes the mean. See Fig. 4 for a concrete example of max pooling.

d) Regularization: A dropout rate of 0.11 was applied after the first activation function layer.

e) Data Augmentation: To improve generalization and reduce overfitting, we utilized a combination of geometric and mixing augmentations. First, we applied Alternating Flipping, Mixup [9] and CutMix [10] data enhancement techniques during training. Both methods combine pairs of training samples and their labels to generate new synthetic examples.

Alternating Flipping applies a deterministic horizontal flip to input images based on their unique index and the current training epoch. Unlike standard random flipping, which relies on stochastic probability, this method ensures that the model

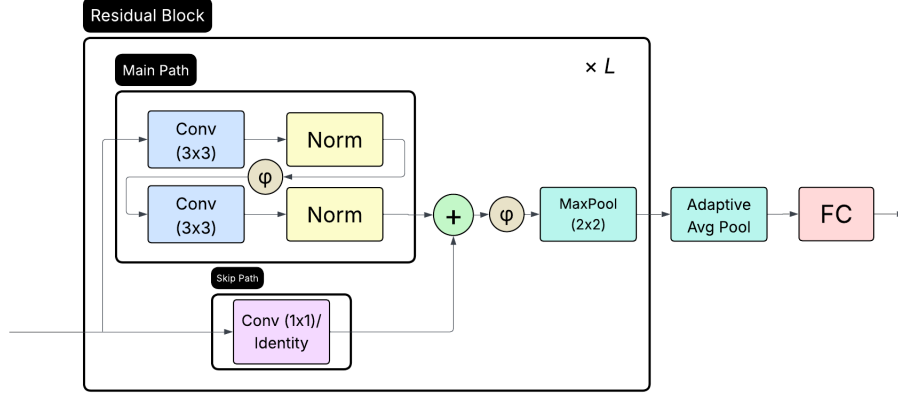


Fig. 2. CNN Architecture. The input is encoded through L -many residual blocks composed of convolutional and pooling layers, where φ denotes an activation function layer. The resulting convolutional feature maps are then aggregated using adaptive average pooling and flattened before being passed into a fully connected linear layer that outputs the final class logits.

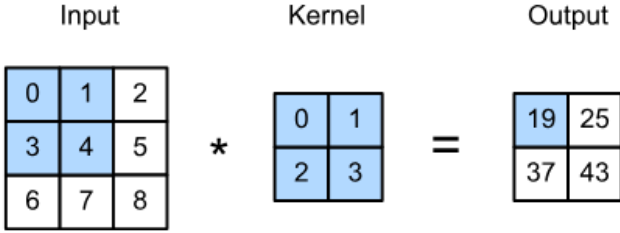


Fig. 3. Convolution with a 2×2 filter and a stride of 1. Figure adapted from [7], [8].

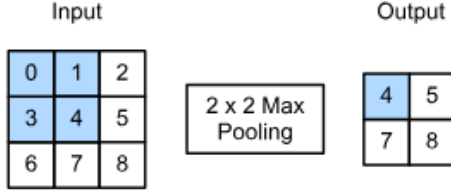


Fig. 4. Max Pooling with a 2×2 filter and a stride of 1. Figure adapted from [7], [8].

encounters both the original and the mirrored version of every sample in a balanced manner over time.

Let x be an input image with a unique dataset index k , and let e denote the current epoch number. We define a hash function $H(k, e)$ that determines the transformation state:

$$\tilde{x} = \begin{cases} \text{flip}(x), & \text{if } (H(k) + e) \equiv 0 \pmod{2}, \\ x, & \text{otherwise,} \end{cases} \quad (6)$$

where $\text{flip}(\cdot)$ represents the horizontal flipping operation and $H(k)$ is a deterministic hash of the sample index. This formulation guarantees that for any given image, the augmentation state toggles between original and flipped in consecutive

epochs, enforcing a uniform exposure to geometric variations without relying on random chance.

Mixup generates a convex combination of two images and their corresponding labels. Given two samples $(\mathbf{X}_i, \mathbf{y}_i)$ and $(\mathbf{X}_j, \mathbf{y}_j)$ and a mixing coefficient $\lambda \sim \text{Beta}(\alpha, \alpha)$, the mixed inputs and labels are computed as:

$$\tilde{\mathbf{X}} = \lambda \mathbf{X}_i + (1 - \lambda) \mathbf{X}_j, \quad \tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1 - \lambda) \mathbf{y}_j. \quad (7)$$

In our experiments, we set $\alpha = 0.4$, which corresponds to sampling λ uniformly from $[0, 1]$.

CutMix replaces a random patch of an image with the corresponding patch from another image. Let M be a binary mask indicating the patch location, then the augmented image and adjusted label are:

$$\tilde{\mathbf{X}} = M \odot \mathbf{X}_i + (1 - M) \odot \mathbf{X}_j, \quad \tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1 - \lambda) \mathbf{y}_j, \quad (8)$$

where λ is adjusted to match the exact pixel proportion of the retained region.

During training, for each batch, one of the augmentation methods (Mixup or CutMix) is applied with probability `mix_prob`; otherwise, standard training is used. Conditional on augmentation being applied, Mixup or CutMix is chosen with equal probability (50/50). The loss for a mixed batch is then computed as a weighted combination of the two target labels:

$$\tilde{\mathcal{L}}(\tilde{\mathbf{X}}, \mathbf{y}_i, \mathbf{y}_j) = \lambda \mathcal{L}(f(\tilde{\mathbf{X}}), \mathbf{y}_i) + (1 - \lambda) \mathcal{L}(f(\tilde{\mathbf{X}}), \mathbf{y}_j), \quad (9)$$

where $\mathcal{L}$ denotes the Cross-Entropy loss as defined in (2) and f denotes the model. For inputs without augmentation, the standard Cross-Entropy loss. For the CNN, we set the label smoothing (3) parameter ϵ to 0.2 for the Cross-Entropy Loss.

f) Learning: The logits were passed to the Cross-Entropy loss in (2) and optimized using Adam with weight decay. To adapt the learning rate during training, we employed PyTorch's `ReduceLROnPlateau` scheduler [11]. In our implementation, the scheduler was configured in `max` mode because it tracked the validation accuracy, and thus the learning rate was

reduced whenever this metric stopped improving. Each reduction halved the current learning rate ($\text{factor} = 0.5$) after 5 consecutive stagnant epochs ($\text{patience} = 5$), allowing the optimizer to take smaller steps once progress plateaued while avoiding premature reductions when the model was still improving.

g) *Hyperparameter Optimization*: To optimize the CNN architecture, we employed a random search strategy over 20 trials. Each trial was trained for a reduced duration of 12 epochs to efficiently identify promising configurations. The search space included learning rates sampled log-uniformly from $[10^{-4}, 3 \times 10^{-3}]$, dropout rates from $[0.1, 0.4]$, and mixup probabilities in $\{0.3, 0.5, 0.7\}$. As illustrated in the correlation analysis (Fig. 5), lower dropout rates were favored, leading us to select a value of 0.11.

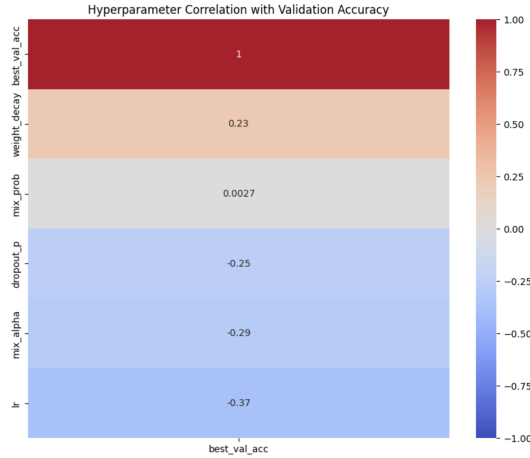


Fig. 5. Hyperparameter Correlation Heatmap. This analysis guided the selection of the final model configuration.

h) *Training Protocol*: The final model was trained for 80 epochs using the Adam optimizer with the best hyperparameters identified during the search (Learning Rate: 2.2×10^{-4} , Weight Decay: 4.7×10^{-4}). We utilized a ReduceLROnPlateau scheduler to dynamically adjust the learning rate when validation accuracy plateaued, ensuring fine-grained weight updates in the later stages of training.

B. Vision Transformer

a) *Architecture*: The Vision Transformer (ViT), divides an input image into non-overlapping $p \times p$ patches, yielding a sequence of $m = HW/p^2$ tokens [12], [13]. In our implementation, patch extraction and projection are performed by a convolutional layer (5) with kernel size and stride p , producing an output of shape $(|\mathcal{B}|, d, H/p, W/p)$. This output is then flattened and transposed to $(|\mathcal{B}|, m, d)$, corresponding to a batch of m tokens each represented by a d -dimensional embedding.

As illustrated in Fig. 6, a learnable class token is concatenated to the sequence of patch embeddings, and learnable positional embeddings are added to all tokens, allowing the model to encode spatial information. The resulting sequence

is then processed by a Transformer encoder block, which includes multi-head self-attention, MLP layers, and Layer Normalization applied before both the attention and MLP sub-layers. Each MLP consists of two linear transformations separated by a GELU activation, following the design introduced in [12].

Finally, the encoded class token is passed through a simple linear layer which outputs class logits. While [12] applied a small MLP head for classification, we replace it with a single linear layer, which has been shown to achieve comparable performance [14].

b) *Self-Attention*: Vision Transformers (ViTs) use self-attention as their core block. Images are divided into patches and linearly embedded, with each patch attending to all others to capture long-range dependencies. Unlike CNNs, self-attention lacks built-in local spatial structure, yielding weaker inductive biases for nearby pixels [12].

Given an input sequence $\mathbf{X} \in \mathbb{R}^{m \times d}$, we introduce three variables: the queries $\mathbf{Q} \in \mathbb{R}^{m \times d_k}$, the keys $\mathbf{K} \in \mathbb{R}^{m \times d_k}$, and the values $\mathbf{V} \in \mathbb{R}^{m \times d_v}$. These matrices represent, for each of the m tokens in the sequence, its query, key, and value vectors that will be used in the attention mechanism. The scaled dot-product attention is defined as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (10)$$

as in [15].

To enhance representational capacity, *multi-head self-attention* introduces learned projection matrices for queries, keys, and values. For each head, we compute:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \quad (11)$$

where $\mathbf{W}_Q \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}_K \in \mathbb{R}^{d \times d_k}$, and $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$ are the weights that project the input into queries, keys, and values respectively.

These learned queries, keys, and values are then passed into several attention operations (10), computed in parallel with distinct projections for each head. The resulting outputs are concatenated and linearly projected, enabling the model to jointly attend to different subspaces (see Fig. 7) [13]. The dimensionality of each head is set by dividing the model dimension d by the number of heads n_{heads} , i.e., $d_k = d/n_{heads}$ [15]. This ensures the concatenated outputs match the original embedding dimension d , while each head learns a distinct portion of the embedding space, improving generalization.

c) *Regularization*: To reduce overfitting, we applied DropPath regularization to each encoder block, randomly dropping residual branches during training. Given an input $\mathbf{x}$ to a residual branch $\mathcal{F}(\mathbf{x})$, the output is

$$\mathbf{y} = \mathbf{x} + b \cdot \mathcal{F}(\mathbf{x}), \quad b \sim \text{Bernoulli}(1 - p), \quad (12)$$

with drop probability $p = 0.1$. In addition, attention and MLP dropout rates of 0.1 were used. This combination of regularization techniques, shown in [16] to be effective for small datasets, helps mitigate overfitting while preserving model capacity.

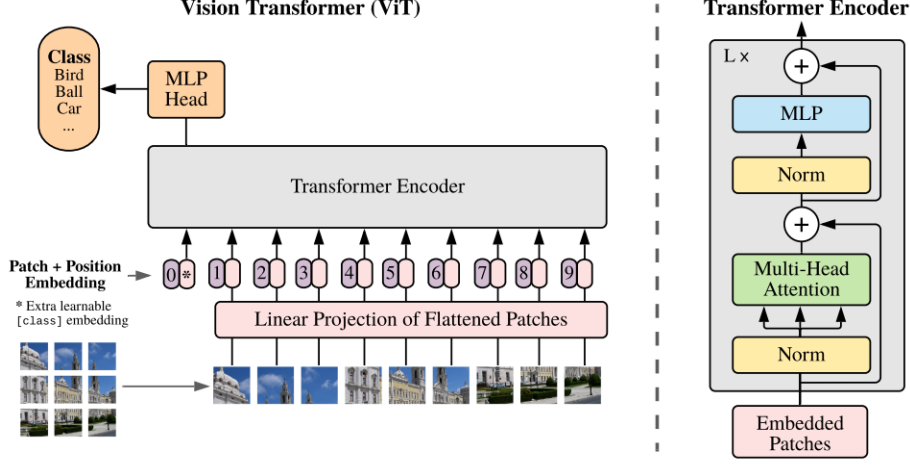


Fig. 6. ViT overview. The image is split into fixed-size patches, linearly embedded with positional encodings, and passed through a Transformer encoder. A learnable classification token enables image classification. Figure derived from [12].

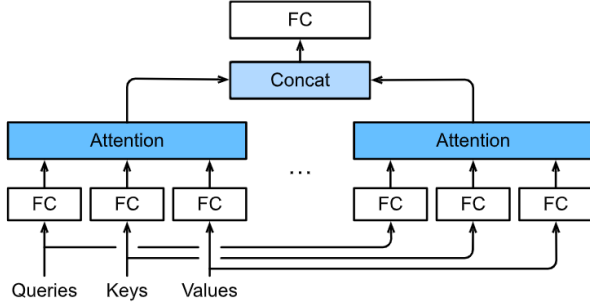


Fig. 7. Multi-Head Attention block. In self-attention, input tokens serve as queries, keys, and values. Each head attends to a different subspace, and outputs are concatenated before a fully connected layer. Figure adapted from [13].

d) Learning: The logits were passed to the Cross-Entropy loss (2) and optimized using Adam with a weight decay of 0.1. Adam was chosen over SGD because it yields better performance in vision transformers [12]. The learning rate was initialized at $\eta_0 = 3 \cdot 10^{-4}$ and updated throughout training using a warm-up-cosine scheduler inspired by [17]. Formally, the learning rate at training step t is

$$\eta_t = \begin{cases} \frac{t}{t_{\text{warmup}}} \eta_0, & 0 \leq t < t_{\text{warmup}}, \\ \frac{\eta_0}{2} \left(1 + \cos \frac{\pi(t-t_{\text{warmup}})}{T-t_{\text{warmup}}} \right), & t_{\text{warmup}} \leq t \leq T, \end{cases} \quad (13)$$

where t_{warmup} is the number of warmup steps and T is the total number of training steps. The learning rate is never allowed to go below a minimum value $\eta_{\min} = 10^{-6}$. In this work, the warmup duration was set to $t_{\text{warmup}} = 0.05 T$, corresponding to 5% of the total training steps.

e) Model Selection: Three transformer encoder configurations were evaluated using stratified 5-fold cross validation. Our custom model was compared against the ViT-Ti and ViT-

S baselines from [12]. The configuration details can be found in Table I. The model with the highest accuracy (4) - averaged over all folds - was ultimately chosen.

Since the total computational cost of cross validation scales as

$$\mathcal{O}(\text{number of configs} \times \text{number of epochs} \times 5), \quad (14)$$

the runtime increases rapidly with the number of hyperparameters explored and epochs trained. Thus, each model was trained for three epochs per fold and only transformer encoder hyper-parameters were cross validated, while all other training hyper-parameters were kept fixed.

After cross validation, the best model was trained for 50 epochs on 80% of the dataset and tested against the other 20%. The test set was evaluated through the Cross-Entropy loss (2) and accuracy (4).

TABLE I
TRANSFORMER ENCODER CONFIGURATIONS

Model	Width	Num. Heads	Depth	MLP Dim
ViT-Custom	256	8	6	512
ViT-Ti	192	3	12	768
ViT-S	384	6	12	1536

C. Ensemble Architecture

a) Architecture: The proposed solution utilizes an ensemble consisting of three models, selected to maximize the different "mistakes" that these models make by making sure to choose ensemble models with varying inductive biases:

- **BiT-M (ResNetV2):** Has the strong inductive bias of traditional CNNs. It relies on locality due to its smaller kernels and translation invariance (recognizing objects regardless of their position). These types of architectures usually focus on more local features in the image.

- **Swin Transformer:** Represents a weaker inductive bias typical of Vision Transformers. Its attention mechanism enables long-range dependencies and capture global context that purely local convolutional filters might miss.
- **ConvNeXt:** Acts as a hybrid bridge that retains the inherent inductive biases of locality and translation invariance found in CNNs with design choices from Transformers. By utilizing larger kernels and architectural tweaks, it achieves a larger effective receptive field than traditional CNNs, allowing it to capture broader context serving as a middle ground in our ensemble.

b) Data Augmentations: **BiT-M** follows the specific “BiT-HyperRule” recipe designed for effective transfer learning. Instead of direct resizing, it uses a `Resize(160)` and `RandomCrop(128)`. This changes the scale of the image to then crop objects from it so that they are not all centered in the images, to improve recognition robustness. The normalization maps pixels to the $[-1, 1]$ range. **Swin Transformer** Lacking the inductive biases of CNNs require regularization to prevent overfitting. We used AutoAugment, a search algorithm to find the augmentations that make a model perform best on a given dataset (CIFAR-10 in our case) [18]. Additionally, RandomErasing randomly applies noise to rectangular patches to each images, discouraging the model to rely on single distinctive features, which was used as data augmentation technique in the original swin ViT Paper [19], [20]. We also apply resizing to 224×224 and flipping to match pre-trained weights [19]. Images were normalized by applying the standard ImageNet normalization. These augmentations are essential for ViTs to generalize well on smaller fine-tuning dataset. For **ConvNeXt:** The pipeline applies RandomAffine to shift images horizontally and vertically by up to 10%. As noted in the ConvNeXt paper, ConvNets’ built-in inductive bias, with the most important one is translation equivariance [4]. This augmentation explicitly reinforces that capability. The images are resized to 224×224 to match the resolution of the pre-trained weights and normalized similarly to the swin ViT.

It is worth noting that both swin ViT and the ConvNeXt use Mixup and CutMix in their training, but we decided to now go ahead with it given our smaller dataset size and that we are fine-tuning on that dataset, and not training from scratch (this is evidenced by Table 6 in [4]).

c) Hyper Parameters: We implemented a Targeted Grid Search to optimize restrict parameter choice to only high-impact parameters (given computational constraints) specific to each architecture’s needs. We performed k-fold cross-validation to determine best hyperparameters to use.

- **BiT-M (ResNetV2):** We adhered to the “BiT-HyperRule” heuristic provided in the paper designed to simplify transfer learning greatly shrinking the search space required hyperparameter tuning [3]. We fixed the optimizer to SGD with Momentum (0.9) and set Weight Decay to 0 and focused our search on Learning Rate Schedules (varying batch size and decay steps) as advised by the paper [3].

- **Swin Transformer:** For the Swin Transformer, we initialized with AdamW (standard for ViTs). We tuned the Drop Path Rate and Learning Rate ($2e-5$, $5e-5$). This strategy aligns with the paper’s emphasis that proper regularization strength is the critical factor for stable fine-tuning and generalization [19].
- **ConvNeXt:** For ConvNeXt, we initialized with the AdamW optimizer. We tuned three ConvNeXt hyperparameters: the learning rate ($\{1e-4, 5e-5\}$), weight decay ($\{0.01, 0.03\}$), and drop path rate ($\{0.1, 0.2\}$). Because our fine-tuning dataset is small, we used slightly lower weight decay values than the paper’s default (0.05) to avoid over-regularization [4].

We did cross-validation on 5 folds for the BiT-M model, but taking into account our computational constraints, we decided to continue with 2 folds for both swin ViT and the ConvNeXt cross-validation. We additionally trained our models using a slightly modified cross-validation to ensure our models see all and different data points (compared to Boot Strapping’s out-of-bag instances for example), this ensures downstream diversity within the same model architecture (each model sees a different fold).

IV. RESULTS

A. CNN

The CNN achieved a final top-1 validation accuracy of 81.25%, significantly outperforming its maximum training accuracy of 74.95%. This inversion—where validation performance exceeds training performance—is attributable to the aggressive data augmentation strategy employed ($\text{mix_prob} = 0.7$). Techniques like Mixup and CutMix construct synthetic, ambiguous training examples with soft targets, making the training objective considerably harder than the evaluation on clean data. This strong regularization effectively prevented the model from memorizing noise in the training set, directly translating to improved generalization on unseen data. The training dynamics, illustrated in Fig. 8, show a rapid convergence during the initial 15 epochs. The stepwise improvements in the loss curve correspond to the scheduler reducing the learning rate at epochs 26, 33, and 51, which allowed the model to escape local minima and refine its decision boundaries without overfitting.

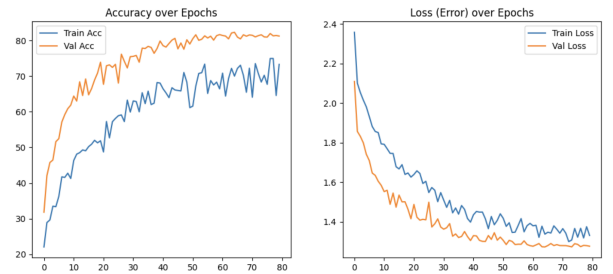


Fig. 8. Training and Validation metrics over 80 epochs. The close tracking of validation accuracy to training accuracy demonstrates the effectiveness of the chosen regularization strategies.

To quantify the number of misclassifications across different categories, we computed the per-class accuracy on the validation set, as visualized in Fig. 9. The histogram reveals that the *Dog* class was the most misclassified, achieving the lowest accuracy of all categories. This high error rate stands in contrast to the strong performance on vehicle classes (e.g., *Ship*, *Automobile*) and is likely attributable to the high intra-class variance and textural similarities shared between dogs and other animal classes, such as cats.

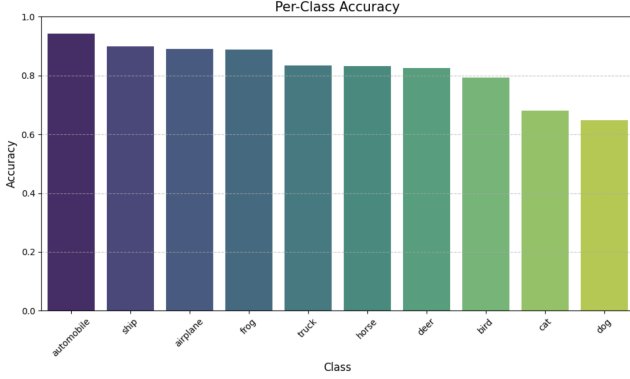


Fig. 9. Histogram of CNN validation accuracies...

B. ViT

The custom ViT configuration achieved the highest performance in stratified 5-fold cross-validation, with an average validation accuracy of 31.28%, outperforming ViT-Ti and ViT-S, which achieved 28.82% and 30.16%, respectively. Thus, the custom configuration was used for further training. custom ViT config After training for 50 epochs, the ViT-Custom model achieved a training accuracy of 69.88% and a test accuracy of 59.50% (see Fig. 10). The generalization gap indicates possible overfitting on the relatively small dataset (10,000 samples). This performance is much better than random guessing in a 10-class setting (10% accuracy), but still limited due to several factors:

- Vision Transformers have weaker inductive biases than CNNs, as they do not inherently encode local spatial relationships. Instead, self-attention mechanisms focus on global dependencies between image patches [12].
- Transformers are highly data hungry and contain more parameters than typical CNNs. For instance, the ViT-Custom model has approximately 3.4M parameters.

Interestingly, the ViT achieves noticeably higher accuracies on vehicle classes compared to animal classes. Which is very similar to that of the CNN (see Fig. 11). A plausible explanation is that many animals share highly similar visual features (e.g., dogs vs. cats, or horses vs. deer), which makes them harder to distinguish. In contrast, vehicles such as ships, airplanes, and automobiles typically occupy a larger portion of the image and exhibit distinctive global structures, which the ViT can capture more effectively. The absence of built-in equivariance and locality in the ViT architecture may further

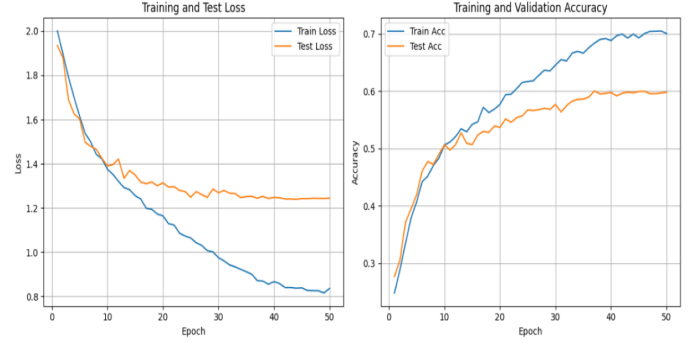


Fig. 10. Near epoch 30, the test loss and accuracies plateau to 1.2 and 60% respectively. This might be due to overfitting on the relatively small (10,000 samples) dataset.

hinder its ability to classify animals when they appear in varying positions or smaller regions of the image.

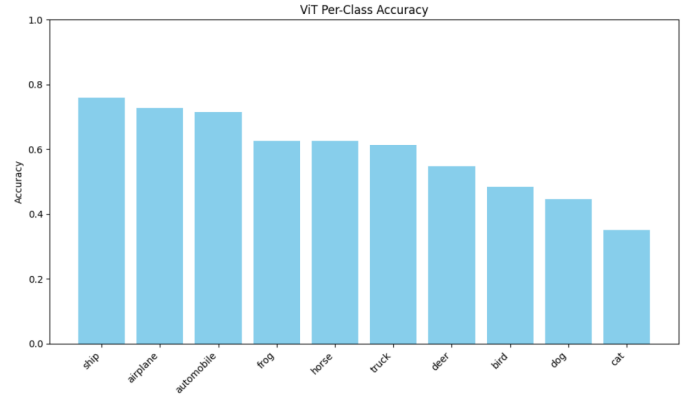


Fig. 11. Histogram of ViT test accuracies across classes. Vehicle categories (ship, airplane, automobile) achieve the highest accuracies, while the truck class is an exception with only 60% accuracy.

C. Ensemble

Our ensemble achieved an accuracy of **97.83%** on the public Kaggle evaluation set (submitted twice), demonstrating strong robustness. Among the individual models, ConvNeXt emerged as the best overall performer, as shown in Fig. 13. Meanwhile, Fig. 12 and Fig. 14 highlight the complementary inductive biases of BiT and Swin-ViT, reinforcing the effectiveness of combining these architectures in an ensemble.

V. CONCLUSION

In conclusion, the CNN achieved a final top-1 validation accuracy of 81.25%, surpassing its maximum training accuracy of 74.95%. This inversion is attributable to aggressive data augmentation (mix_prob = 0.7), where Mixup and CutMix produced synthetic, ambiguous training examples with soft targets. Such strong regularization prevented memorization of noise and improved generalization, as reflected in the rapid convergence during the first 15 epochs and stepwise loss improvements following scheduled learning rate reductions (epochs 26, 33, and 51). Per-class validation accuracy (Fig. 9)

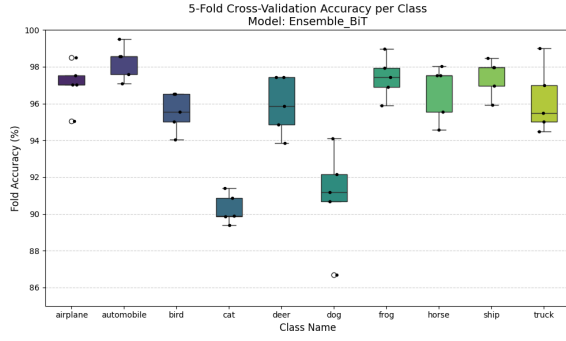


Fig. 12. BiT Box Plot

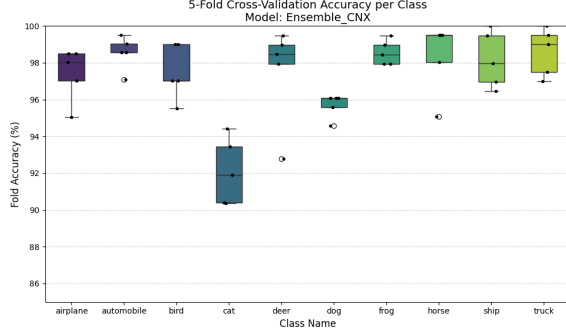


Fig. 13. ConvexNet Box Plot

revealed that the *Dog* class was most misclassified, likely due to high intra-class variance and textural similarities with other animals, whereas vehicle classes (e.g., *Ship*, *Automobile*) achieved strong performance.

By contrast, the Vision Transformer (ViT) achieved a training accuracy of 69.88% and a test accuracy of 59.50%. Its lower performance stems from weaker inductive biases for local features, a smaller parameter count (3.4M vs. 14M for CNN), and limited training (50 epochs). While CNNs excel at local feature detection via convolution (5), ViTs rely on global dependencies through attention (10). Training dynamics (Fig. 10) indicate overfitting, with validation accuracy plateauing despite rising training accuracy. To mitigate this, future work should employ stronger augmentation (e.g., Mixup (7),

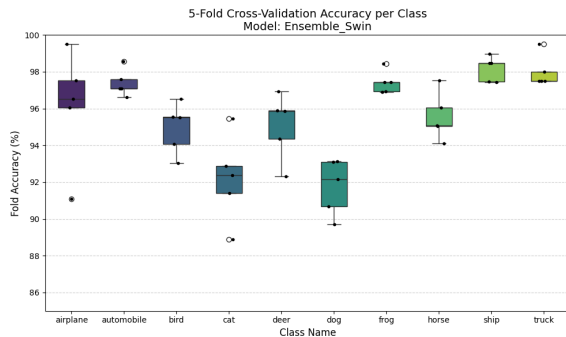


Fig. 14. swin Box Plot

CutMix (8)), increased regularization (dropout, weight decay), and label smoothing (3) prior to Cross-Entropy loss (2). Moreover, scaling ViT's parameter count while applying these regularization strategies may yield improved generalization and performance.

Finally, our ensemble achieved an accuracy of **97.83%** on the public Kaggle evaluation set, demonstrating strong robustness. As shown in Fig. 12, Fig. 13, and Fig. 14, the weaker performance of the transformer-based models (Swin Transformer and BiT) suggests that they would benefit from longer training.

VI. STATEMENT OF CONTRIBUTIONS

Marc Germanos designed the CNN. Parsa Jafarian built the ViT. Nicolaas Saade implemented the ensemble method that combined ConvNeXt, Swin Transformer, and BiT models.

REFERENCES

- [1] M. Coates, "Lecture 12: Backpropagation and training neural networks (murphy 13)." ECSE 551 – Machine Learning for Engineers, McGill University, Montreal, QC, Canada, Oct. 2025.
- [2] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," 2015.
- [3] A. Kolesnikov, L. Beyer, X. Zhai, J. Puigcerver, J. Yung, S. Gelly, and N. Houlsby, "Big transfer (bit): General visual representation learning," 2020.
- [4] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," 2022.
- [5] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," *CoRR*, vol. abs/2103.14030, 2021.
- [6] MathWorks, "Adaptiveaveragepooling2dlayer." <https://www.mathworks.com/help/deeplearning/ref/nnet.cnn.layer.adaptiveaveragepooling2dlayer.html>, 2025. Accessed: 2025-12-03.
- [7] M. Coates, "Lecture 14: Convolutional neural networks (murphy 14)." ECSE 551 – Machine Learning for Engineers, McGill University, Montreal, QC, Canada, Nov. 2025.
- [8] K. P. Murphy, *Probabilistic Machine Learning: An introduction*. MIT Press, 2022.
- [9] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," 2018.
- [10] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "Cutmix: Regularization strategy to train strong classifiers with localizable features," 2019.
- [11] PyTorch, "torch.optim.lr_scheduler.reduceLronplateau." https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html, 2025. Accessed: 2025-11-30.
- [12] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021.
- [13] M. Coates, "Lecture 17: Transformers." ECSE 551 – Machine Learning for Engineers, McGill University, Montreal, QC, Canada, Nov. 2025.
- [14] L. Beyer, X. Zhai, and A. Kolesnikov, "Better plain vit baselines for imagenet-1k," 2022.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023.
- [16] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer, "How to train your vit? data, augmentation, and regularization in vision transformers," 2022.
- [17] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," 2017.
- [18] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "Autoaugment: Learning augmentation policies from data," 2019.
- [19] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," 2021.
- [20] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," 2017.